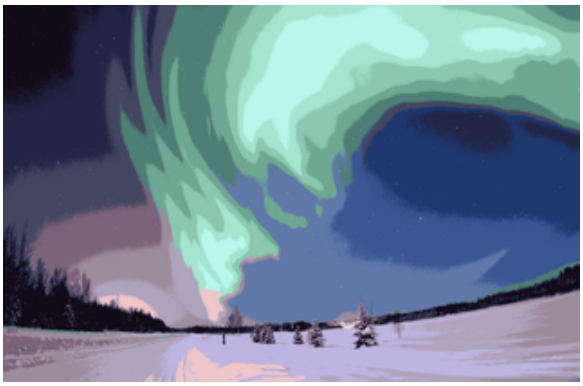
Image segmentation with Convolutional Neural Networks

Image segmentation is a classic task in computer vision. The goal is to group the pixels in an image according to a given criterion. Segmentation finds application in many domains, from medical imaging to industrial inspection, autonomous driving, and more.

Classic methods for image segmentation group pixels into regions of uniform color or texture. Many of these methods use clustering algorithms.

→ 

More recently, **semantic** segmentation became a very popular topic. In semantic segmentation, the regions reflect the content of the image. Ideally, each segment represents one kind of object in the scene.

↓

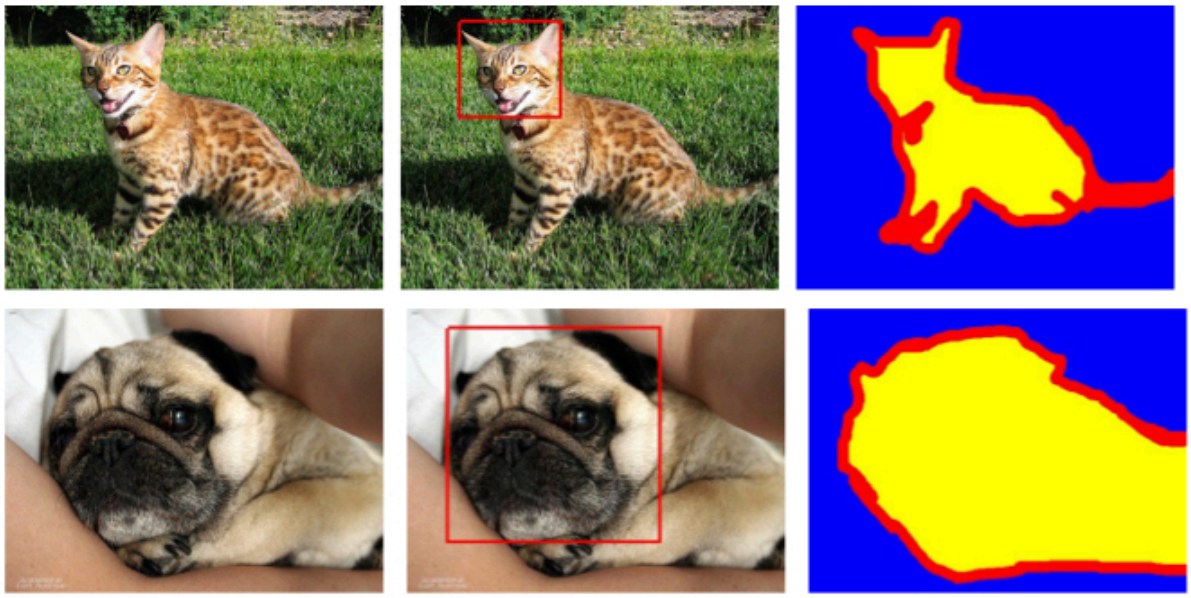
Semantic segmentation entails a pixel-level classification, where each pixel is labeled with one class taken from a set of predefined classes.

Convolutional Neural Networks (CNN) are good candidates for solving this problem. A CNN may analyze the whole image and then output a vector of probability estimates for each pixel. Segmentation is achieved by labeling pixels as the class corresponding to the highest estimate.

Pet segmentation

In this exercise, we will address a relatively simple semantic segmentation problem. More precisely, we want to build a model capable of segmenting images of pets.

We will use the [Oxford Pet Dataset](), which includes 7349 images of cats and dogs, each with a manually defined segmentation mask.



Here, we have three classes:

- class 0 is the background of the picture;
- class 1 is a pixel of a cat;

- class 2 is a pixel of a dog.

The dataset also includes bounding boxes, which are useful for training models for object detection but which we won't use.

Setup and Data

The dataset is distributed as pairs of files representing an image of a pet and the corresponding segmentation map. Files are organized in a tree of directories with the following structure:

- the root (`pets`) includes four directories: `train_images`, `train_maps`, `test_images`, `test_maps`;
- each folder includes 37 folders representing different breeds of cats and dogs (the breed is not relevant information for this project);
- each breed folder contains a set of image files: .jpg for the images, .png for the segmentation maps.

Folders containing images and maps have the same structure, so it is possible to pair their content. For instance, the files

- `pets/test_images/Abyssinian/Abyssinian_105.jpg`
- and `pets/test_maps/Abyssinian/Abyssinian_105.png` form an image/map pair.

This structure allows the simple definition of a torch `Dataset` class, yielding pairs of corresponding images and maps.

Images are encoded as RGB JPEG-compressed files and can be processed by standard tools in the `torchvision` module.

Maps are grayscale PNG-compressed files. Each pixel in the segmentation map may assume the values 0, 100, and 200 representing the classes background, cat and dog. We will need to convert RGB (which is the torchvision default) maps to grayscale and map the values in the 0, 1, 2 integers we will use as class labels.

```
import torch
import numpy as np
import torchvision
import matplotlib.pyplot as plt
import google.colab
import zipfile
```

```
google.colab.drive.mount('/content/drive')
PROJECT_DIR = "/content/drive/MyDrive/Colab Notebooks/"
zip_ref = zipfile.ZipFile(PROJECT_DIR + "pet-segmentation .zip", 'r')
zip_ref.extractall(".")
zip_ref.close()
print("OK")
```

```
Mounted at /content/drive
OK
```

To manage the data, we will define our own `Dataset`, which will include two sub-datasets: one for the RGB images and the other for the segmentation maps. Our dataset must correctly pair

the two kinds of images.

```
class PetsDataset(torch.utils.data.Dataset):
    def __init__(self, root, train, im_transformation, map_transformation):
        prefix = root + ("/train_" if train else "/test_")
        images_dir = prefix + "images"
        self.images = torchvision.datasets.ImageFolder(images_dir, transform = im_tran
        maps_dir = prefix + "maps"
        self.maps = torchvision.datasets.ImageFolder(maps_dir, transform = map_transfo

    def __len__(self):
        return len(self.images)

    def __getitem__(self, index):
        images = self.images[index][0]
        maps = self.maps[index][0]
        return (images, maps)


MEAN_RGB = [0.4849, 0.4495, 0.3966]
STDDEV_RGB = [0.2631, 0.2584, 0.2652]

image_transform  = torchvision.transforms.Compose([
    torchvision.transforms.ToTensor(),
    torchvision.transforms.Normalize(MEAN_RGB, STDDEV_RGB)
])

def process_map(mapdata):
    # RGB to Grayscale and conversion to the integer labels 0, 1, 2.
    return (mapdata[0:1, :, :] * 255 / 100).long() # scales pixel values to the range

map_transform  = torchvision.transforms.Compose([
    torchvision.transforms.ToTensor(),
    process_map
])

train_dataset = PetsDataset("pets", True, image_transform, map_transform)
train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=16, shuffle=True,
test_dataset = PetsDataset("pets", False, image_transform, map_transform)
test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=16, shuffle=True, d

def show_batch(images, maps, out=None):
    plt.rcParams['figure.dpi'] = 200
    g = torchvision.utils.make_grid(images, normalize = True, value_range=(-1, 1))
    plt.imshow(g.permute(1, 2, 0).detach().cpu().numpy())
    plt.axis('off')
    if out is not None:
        plt.figure()
        g = torchvision.utils.make_grid(out)
        plt.imshow(g[0:1].permute(1, 2, 0).detach().cpu(), interpolation ="nearest", c
        plt.axis('off')
    plt.figure()
    g = torchvision.utils.make_grid(maps)
    plt.imshow(g[0:1].permute(1, 2, 0).detach().cpu(), interpolation="nearest", cmap =
    plt.axis('off')
    plt.show()

images, maps =next(iter(train_loader)) # Load one batch of images + masks
print(images)       # Show raw tensor values
print(images.shape) # Show batch dimensions
```

```
show_batch(images, maps)
```

Show hidden output

Convolutional Networks for semantic segmentation

A typical CNN for image classification includes two main components. First, convolutional layers extract local features from the input image. The number of features increases layer after layer. At the same time, pooling layers reduce the spatial resolution. At some point, global pooling (or flattening) reduces the local features to a single vector. The second component is a multi-layer perceptron (MLP) made of fully connected layers, ending with a vector of probability estimates. We can visualize the shape of such a network as a pyramid.

Convolutional Neural Networks can be used for segmentation as well. The MLP component is removed to preserve spatial information. Instead, a decoder is introduced to restore the original resolution and perform a pixel-level classification. The first example of this approach is the Fully Convolutional Network (FCN), described in the paper:

J. Long, E. Shelhamer and T. Darrel, "Fully Convolutional Networks for Semantic Segmentation," CVPR 2015 (PDF).

Here, we will define a symmetric encoder/decoder network. The *Encoder* is like the convolutional part of a CNN for image classification. The *Decoder* acts specularly by progressively restoring the original spatial resolution and reducing the number of features. This network would have the shape of an hourglass.

Each step is obtained by a *convolutional block* including two convolutions, normalization layers, and ReLUs. It also optionally applies max pooling (to reduce spatial resolution) and upscale layers (to increase it). The convolutional block is implemented as a pytorch `Module`:

```
class ConvBlock(torch.nn.Module):
    def __init__(self, inchannels, outchannels, downscale=False, upscale=False):
        super(ConvBlock, self).__init__()
        self.down = torch.nn.MaxPool2d(2) if downscale else torch.nn.Identity()
        self.conv1 = torch.nn.Conv2d(inchannels, outchannels, 3, padding=1)
        self.bnorm1 = torch.nn.BatchNorm2d(outchannels)
        self.conv2 = torch.nn.Conv2d(outchannels, outchannels, 3, padding=1)
        self.bnorm2 = torch.nn.BatchNorm2d(outchannels)
        self.up = torch.nn.Upsample(scale_factor=2) if upscale else torch.nn.Identity(

    def forward(self, x):
        x = self.down(x)
        x = torch.nn.functional.relu(self.bnorm1(self.conv1(x)))
        x = torch.nn.functional.relu(self.bnorm2(self.conv2(x)))
        x = self.up(x)
        return x
```

Blocks are assembled to define the FCN. We will follow the scheme:

$$\text{input} \xrightarrow{\textit{Encoder}} \cdot \xrightarrow{\textit{Bottleneck}} \cdot \xrightarrow{\textit{Decoder}} \text{output}$$

where:

- The Encoder is a sequence of convolutional blocks with max pooling;
- The Decoder is a sequence of convolutional blocks with upsampling;
- The Bottleneck is a single convolution block (with both pooling and upsampling).

```
class EncoderDecoder(torch.nn.Module):
    def __init__(self, classes, depth):
        super(EncoderDecoder, self).__init__()
        self.encoder = torch.nn.ModuleList()
        channels = [3] + [64 * (2 ** i) for i in range(depth)]
        # Encoder
        for i in range(depth):
            self.encoder.append(
                ConvBlock(channels[i], channels[i + 1], downscale=(i > 0))
            )
        # Bottleneck
        self.bottleneck = ConvBlock(channels[-1], channels[-1], downscale=True, upscal
        # Decoder
        self.decoder = torch.nn.ModuleList()
        channels[0] = 64  # fix first channel for decoder
        for i in range(depth - 1, -1, -1):
            self.decoder.append(
                ConvBlock(channels[i + 1], channels[i], upscale=(i > 0))
            )
        # Classifier
        self.classifier = torch.nn.Conv2d(channels[0], classes, 1)
    def forward(self, x):
        for mod in self.encoder:
            x = mod(x)
        x = self.bottleneck(x)
        for mod in self.decoder:
            x = mod(x)
        x = self.classifier(x)
        return x
net = EncoderDecoder(3, 3)
print(sum(p.numel() for p in net.parameters()), "parameters")

x = torch.zeros(1, 3, 160, 160)
print(net(x).shape)
```

```
2956803 parameters
torch.Size([1, 3, 160, 160])
```

Training loop

Since semantic segmentation can be seen as pixel-level classification, we can use cross entropy as loss function. We just need to average it over all pixels in the batch of images. Beside that, the training loop has no special features.

```
from torch.nn.modules import loss
EPOCHS = 10
LR = 0.001
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
print("Device:", DEVICE)
net.to(DEVICE)
```

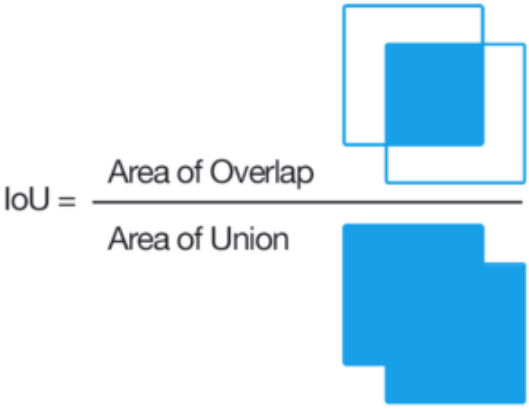
```
optimizer = torch.optim.Adam(net.parameters(), LR)
net.train()
steps = 0
for epoch in range(EPOCHS):
    for images, labels in train_loader:
        images = images.to(DEVICE)
        labels = labels.to(DEVICE)
        steps += 1
        out = net(images)
        optimizer.zero_grad()
        map = net(images)
        loss = torch.nn.functional.cross_entropy(map, labels.squeeze(1))
        loss.backward()
        optimizer.step()
        if steps % 50 ==0:
          predictions = map.argmax(1)
          accuracy = 100 * (predictions == labels.squeeze(1)).float().mean()
          print(f"{steps} [{epoch}] loss: {loss.item():.4f} accuracy: {accuracy.item()

    data ={
        "net": net.state_dict(),
        "optimizer": optimizer.state_dict()
    }
    torch.save(data, PROJECT_DIR + "segmentation.pt")
```

```
1100 [2] loss: 0.5531 accuracy: 75.29%
1150 [2] loss: 0.5474 accuracy: 77.41%
1200 [3] loss: 0.6122 accuracy: 75.08%
1250 [3] loss: 0.5313 accuracy: 77.93%
1300 [3] loss: 0.5133 accuracy: 78.95%
1350 [3] loss: 0.4265 accuracy: 83.19%
1400 [3] loss: 0.4551 accuracy: 83.50%
1450 [3] loss: 0.4521 accuracy: 81.46%
1500 [3] loss: 0.4817 accuracy: 79.81%
1550 [3] loss: 0.5754 accuracy: 74.93%
1600 [4] loss: 0.4595 accuracy: 80.54%
1650 [4] loss: 0.4085 accuracy: 82.93%
1700 [4] loss: 0.4549 accuracy: 80.23%
1750 [4] loss: 0.5528 accuracy: 75.50%
1800 [4] loss: 0.4674 accuracy: 82.15%
1850 [4] loss: 0.5475 accuracy: 77.96%
1900 [4] loss: 0.4239 accuracy: 82.42%
```

```
3050 [7] loss: 0.3234 accuracy: 87.12%
3100 [7] loss: 0.3527 accuracy: 85.48%
3150 [7] loss: 0.3683 accuracy: 86.39%
3200 [8] loss: 0.3136 accuracy: 88.64%
3250 [8] loss: 0.4872 accuracy: 79.35%
3300 [8] loss: 0.5519 accuracy: 78.02%
3350 [8] loss: 0.3860 accuracy: 84.72%
3400 [8] loss: 0.3493 accuracy: 87.03%
3450 [8] loss: 0.3481 accuracy: 86.44%
3500 [8] loss: 0.3843 accuracy: 84.45%
3550 [8] loss: 0.4029 accuracy: 82.98%
3600 [9] loss: 0.2453 accuracy: 90.30%
3650 [9] loss: 0.6016 accuracy: 76.97%
3700 [9] loss: 0.2676 accuracy: 89.58%
3750 [9] loss: 0.4294 accuracy: 83.86%
3800 [9] loss: 0.2893 accuracy: 89.85%
3850 [9] loss: 0.3720 accuracy: 84.97%
3900 [9] loss: 0.2470 accuracy: 91.68%
3950 [9] loss: 0.3866 accuracy: 83.89%
```

Evaluation

We can use different statistics to measure performance. Accuracy is the most straightforward (fraction of correctly classified pixels). A common alternative is the "Intersection over Union" (IoU).



It works like this:

1. for each class, measure the intersection area between the pixels of that class and those predicted as such.
2. Repeat, this time computing the union instead of the intersection.
3. Compute the ratio.
4. Average the ratios obtained for all the classes.

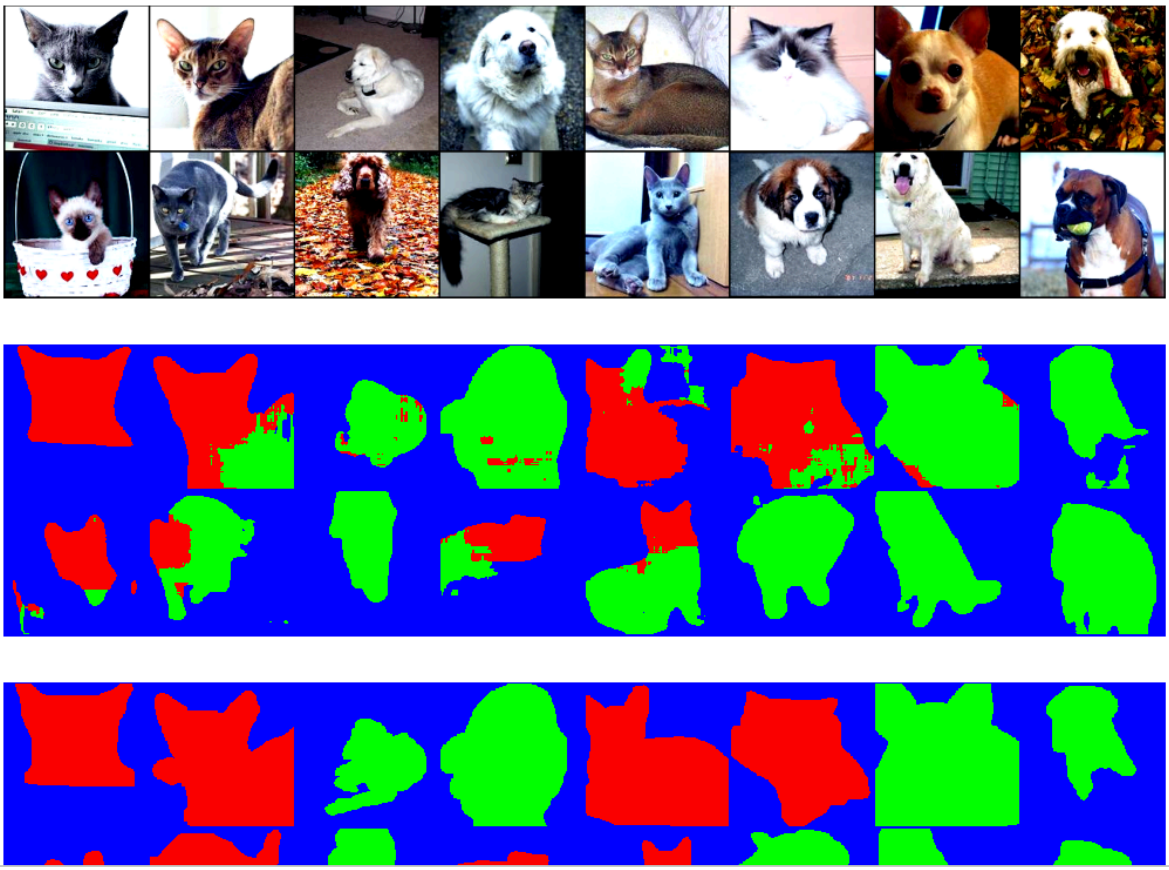
IoU ranges between zero (worse) and one (perfect). The advantage of IoU over accuracy is that IoU is not influenced by the number of correctly classified "negative" pixels, which is often misleading.

We can easily compute IoU from the confusion matrix.

```
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
print("Device:", DEVICE)
data = torch.load(PROJECT_DIR + "segmentation.pt", map_location=DEVICE)
net.to(DEVICE)
net.load_state_dict(data["net"])
#optimizer.load_state_dict(data["optimizer"])
```

```
Device: cuda
<All keys matched successfully>
```

```
net.eval()
im, map = next(iter(test_loader))
out = net(im.to(DEVICE)).argmax(1, True).to("cpu")
show_batch(im, map, out)

classes = 3
confusion_matrix = torch.zeros(classes ** 2, dtype=int)
for images, labels in test_loader:
  images = images.to(DEVICE)
  labels = labels.to(DEVICE)
  with torch.no_grad():
    map = net(images)
    predictions = map.argmax(1)
    entries = predictions * classes + labels.squeeze(1)
    counts = torch.bincount(entries.view(-1), minlength=classes ** 2)
    confusion_matrix += counts.cpu()
```



```
confusion_matrix = confusion_matrix.view(classes, classes)
normalized = confusion_matrix / confusion_matrix.sum(0, True)
print("CONFUSION MATRIX")
for row in normalized.tolist():
  print(("{:.3f}" * len(row)).format(*row))

print()
```

```
accuracy  = 100 * (confusion_matrix.diag().sum() / confusion_matrix.sum()).item()
iou = 100 * (confusion_matrix.diag() / (confusion_matrix.sum(1) + confusion_matrix.sum
print("ACCURACY:", accuracy)
```

```
CONFUSION MATRIX
0.9260.1150.083
0.0150.5600.029
0.0590.3250.887

ACCURACY: 85.05131006240845
```

UNET

U-Net is a convolutional neural network that was developed for biomedical image segmentation. It is based on a symmetric Encoder/Decoder CNN, with additional *skip connections* to preserve information at different resolutions. Ronneberger et al. presented in:

O. Ronneberger, P. Fischer and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," MICCAI 2015 (PDF).

In the Decoder, the incoming features are concatenated to the features computed by the symmetric block in the Encoder. This design better preserves spatial information at all scales.

The following scheme, taken from the original paper, shows the architecture. Note that, in this particular case, input and output resolution are not identical (which is unusual). here.

```
class UNet(torch.nn.Module):
    def __init__(self, classes, depth):
        super(UNet, self).__init__()
        #Encoder
        self.encoder = torch.nn.ModuleList()
        channels =[3] + [64 * (2 ** i) for i in range(depth)]
        for i in range(depth):
            self.encoder.append(
                ConvBlock(channels[i], channels[i+1], downscale=(i>0))
            )
        #bottleneck
        self.bottleneck = ConvBlock(channels[-1], channels[-1], downscale=True, upscal
        #Decoder
        self.decoder = torch.nn.ModuleList()
        channels[0] = 64
        for i in range(depth-1, -1, -1):
            self.decoder.append(
                ConvBlock(2 * channels[i+1], channels[i], upscale=(i>0))
            )
        # Final classifier
        self.classifier = torch.nn.Conv2d(64, classes, 1)

    def forward(self, x):
        skip = []
        #Encoder
        for mod in self.encoder:
            x = mod(x)
            skip.append(x)
        #Bottleneck
        x= self.bottleneck(x)
        #Decoder
        for mod in self.decoder:
```

```
            y = skip.pop()
            x= torch.cat([x, y], 1)
            x = mod(x)
        #final classifier
        x = self.classifier(x)
        return x
net = UNet(3, 3)
print(sum(p.numel() for p in net.parameters()), "parameters")

x = torch.zeros(1, 3, 160, 160)
print(net(x).shape)
```

```
3362307 parameters
torch.Size([1, 3, 160, 160])
```

Questions

In the report, answer to the following questions:

- Identify badly segmented test images and group them according to the most likely cause for the errors.
- Find cases in which accuracy and IoU disagree the most (not in absolute value!).

Extensions

The models presented here are by no means optimal. Try to identify the best combination of parameters, also by adjusting the number of features, blocks etc.

Other possible investigations:

- Try data augmentation, such as the photometric transforms described here. If you want to try to use geometric transforms as well, you need to find a way to apply the same transformation to both the input image and the segmentation mask.
- Try introducing dropout layers to improve generalization.
- Replace max pooling with strided convolutions (set stride = 2 in Conv2d).
- Replace upscaling with strided transposed convolutions.

```
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"

# Load the trained EncoderDecoder model (NOT UNet)
net = EncoderDecoder(3, 3).to(DEVICE)
data = torch.load(PROJECT_DIR + "segmentation.pt", map_location=DEVICE)
net.load_state_dict(data["net"])
net.eval()
# Metric functions
def compute_metrics(pred, gt, num_classes=3):
    accuracy = (pred == gt).float().mean().item()
    ious = []
    for c in range(num_classes):
        intersection = ((pred == c) & (gt == c)).sum().float()
        union = ((pred == c) | (gt == c)).sum().float()
        if union > 0:
            ious.append((intersection / union).item())
    mean_iou = sum(ious) / len(ious)
    return accuracy, mean_iou
# Collect per-image statistics
```

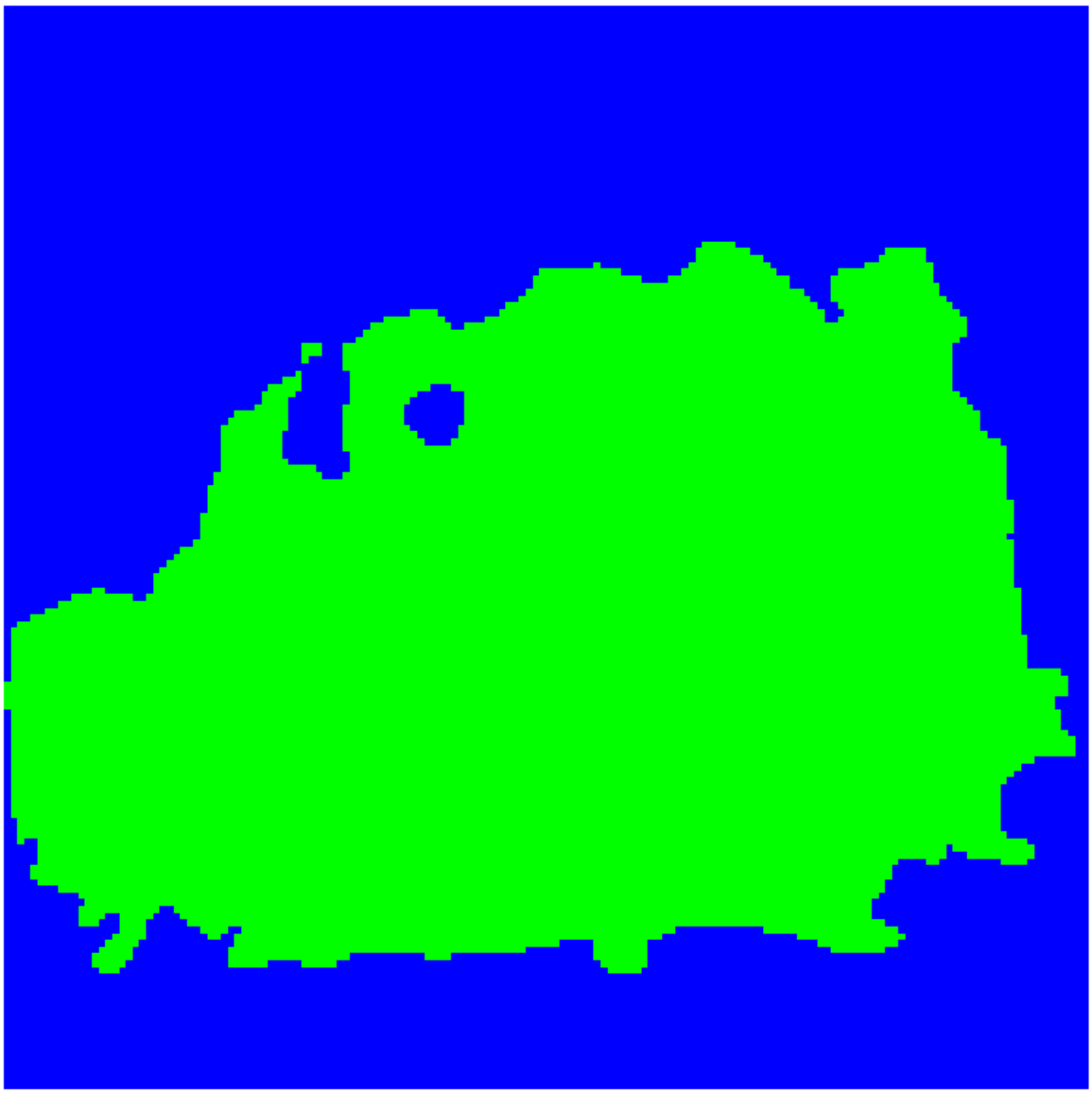
```
image_stats = []

with torch.no_grad():
    for images, labels in test_loader:
        images = images.to(DEVICE)
        labels = labels.to(DEVICE)
        outputs = net(images)
        preds = outputs.argmax(1)
        for i in range(images.size(0)):
            acc, miou = compute_metrics(preds[i], labels[i, 0])
            image_stats.append((acc, miou, images[i].cpu(), labels[i].cpu

# Sort by worst IoU (bad segmentation)
bad_cases = sorted(image_stats, key=lambda x: x[1])  # sort by IoU ascend
# display images
def show_single_case(img, gt, pred):
    plt.figure(figsize=(12, 4))
    plt.subplot(1, 3, 1)
    plt.title("Image")
    plt.imshow(img.permute(1, 2, 0).numpy())
    plt.axis("off")
    plt.subplot(1, 3, 2)
    plt.title("Ground Truth")
    plt.imshow(gt[0], cmap="brg")
    plt.axis("off")
    plt.subplot(1, 3, 3)
    plt.title("Prediction")
    plt.imshow(pred, cmap="brg")
    plt.axis("off")

    plt.show()

# Automatic grouping of error causes
def classify_error(img, gt, pred):
    # Very low IoU → likely confusion or wrong class
    if (pred != gt[0]).float().mean() > 0.7:
        return "Severe misclassification (model confused)"
    # If pet is small in GT but prediction is large
    pet_pixels_gt = (gt[0] > 0).sum().item()
    pet_pixels_pred = (pred > 0).sum().item()
    if pet_pixels_gt < 5000:
        return "Small pet → model struggles with tiny objects"
    # If background is predicted as pet
    if pet_pixels_pred > pet_pixels_gt * 2:
        return "Background confusion (fur color similar to background)"
    # If edges are messy
    boundary_errors = ((pred != gt[0]) & (gt[0] > 0)).float().mean().item
    if boundary_errors > 0.4:
        return "Boundary/shape errors (complex pose or fur texture)"
    return "Mild segmentation error"

# Display top N worst cases with categories
N = 10  # number of bad examples to show
for idx in range(N):
    acc, miou, img, gt, pred = bad_cases[idx]
    category = classify_error(img, gt, pred)
    print(f"\nBAD CASE #{idx+1}")
    print(f"Accuracy: {acc:.3f}, IoU: {miou:.3f}")
    print(f"Likely cause: {category}")
    show_single_case(img, gt, pred)
```

Show hidden output

```
import torch
import torch.nn as nn
import torch.nn.functional as F

# Basic Conv Block with Dropout
class ConvBlock(nn.Module):
    def __init__(self, in_channels, out_channels, dropout_p=0.3):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=3,
        self.bnorm1 = nn.BatchNorm2d(out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3,
        self.bnorm2 = nn.BatchNorm2d(out_channels)
        self.dropout = nn.Dropout2d(p=dropout_p)
    def forward(self, x):
        x = F.relu(self.bnorm1(self.conv1(x)))
        x = F.relu(self.bnorm2(self.conv2(x)))
        x = self.dropout(x)
        return x
# Down Block: Strided Conv instead of MaxPool
class DownBlock(nn.Module):
    def __init__(self, in_channels, out_channels, dropout_p=0.3):
        super().__init__()
        # Strided conv for downsampling
        self.down = nn.Conv2d(in_channels, in_channels, kernel_size=3, st
        self.conv = ConvBlock(in_channels, out_channels, dropout_p=dropou
    def forward(self, x):
        x = self.down(x)
        x = self.conv(x)
        return x
# Up Block: Transposed Conv instead of Upsample
class UpBlock(nn.Module):
    def __init__(self, in_channels, out_channels, dropout_p=0.3):
        super().__init__()
        # Transposed conv for upsampling
        self.up = nn.ConvTranspose2d(in_channels, in_channels // 2, kerne
        # After concatenation: in_channels = (in_channels // 2) + skip_ch
        self.conv = ConvBlock(in_channels, out_channels, dropout_p=dropou
    def forward(self, x, skip):
        x = self.up(x)
        # Pad if needed (in case of odd sizes)
        diff_y = skip.size(2) - x.size(2)
        diff_x = skip.size(3) - x.size(3)
        x = F.pad(x, [diff_x // 2, diff_x - diff_x // 2,
                      diff_y // 2, diff_y - diff_y // 2])
        x = torch.cat([skip, x], dim=1)
        x = self.conv(x)
        return x

# Full UNet with:
class UNet(nn.Module):
    def __init__(self, in_channels=3, num_classes=3, base_channels=64, dr
        super().__init__()

        # Encoder
        self.enc1 = ConvBlock(in_channels, base_channels, dropout_p=dropo
        self.enc2 = DownBlock(base_channels, base_channels * 2, dropout_p
        self.enc3 = DownBlock(base_channels * 2, base_channels * 4, dropo
        self.enc4 = DownBlock(base_channels * 4, base_channels * 8, dropo
        # Bottleneck
```

```
        # Bottleneck
        self.bottleneck = DownBlock(base_channels * 8, base_channels * 16
        # Decoder
        self.dec1 = UpBlock(base_channels * 16, base_channels * 8, dropou
        self.dec2 = UpBlock(base_channels * 8, base_channels * 4, dropout_
        self.dec3 = UpBlock(base_channels * 4, base_channels * 2, dropout_
        self.dec4 = UpBlock(base_channels * 2, base_channels, dropout_p=d
        # Final classifier
        self.classifier = nn.Conv2d(base_channels, num_classes, kernel_si
    def forward(self, x):
        # Encoder
        x1 = self.enc1(x)          # (B, 64, H, W)
        x2 = self.enc2(x1)         # (B, 128, H/2, W/2)
        x3 = self.enc3(x2)         # (B, 256, H/4, W/4)
        x4 = self.enc4(x3)         # (B, 512, H/8, W/8)
        # Bottleneck
        xb = self.bottleneck(x4)   # (B, 1024, H/16, W/16)
        # Decoder
        x = self.dec1(xb, x4)      # (B, 512, H/8, W/8)
        x = self.dec2(x, x3)       # (B, 256, H/4, W/4)
        x = self.dec3(x, x2)       # (B, 128, H/2, W/2)
        x = self.dec4(x, x1)       # (B, 64, H, W)
        # Output
        x = self.classifier(x)     # (B, num_classes, H, W)
        return x
```

```
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"

net = UNet(in_channels=3, num_classes=3).to(DEVICE)
print("Model ready on:", DEVICE)
EPOCHS = 2
LR = 0.001
optimizer = torch.optim.Adam(net.parameters(), lr=LR)
net.train()
for epoch in range(EPOCHS):
    for images, labels in train_loader:
        images = images.to(DEVICE)
        labels = labels.to(DEVICE)
        optimizer.zero_grad()
        outputs = net(images)
        loss = torch.nn.functional.cross_entropy(outputs, labels.squeeze(1))
        loss.backward()
        optimizer.step()

    print(f"Epoch {epoch+1}/{EPOCHS} - Loss: {loss.item():.4f}")
```

```
Model ready on: cuda
Epoch 1/2 - Loss: 0.7896
Epoch 2/2 - Loss: 0.6942
```

```
net.eval()

images, labels = next(iter(test_loader))
images = images.to(DEVICE)
labels = labels.to(DEVICE)
with torch.no_grad():
    outputs = net(images)
    preds = outputs.argmax(1)
show_batch(images.cpu(), labels.cpu(), preds.cpu())
```